Assessing the Impact of Code Changes on the Fault Localizability of Large Language Models

1st Sabaat Haroon
Virginia Tech
USA
sabaat@vt.edu

2nd Ahmad Faraz Khan
Virginia Tech
USA
ahmadfk@vt.edu

3rd Ahmad Humayun
Virginia Tech
USA
ahmad35@vt.edu

4th Waris Gill
Virginia Tech
USA
waris@vt.edu

5th Abdul Haddi Amjad
Virginia Tech
USA
hadiamjad@vt.edu

6th Ali R. Butt
Virginia Tech
USA
butta@vt.edu

7th Mohammad Taha Khan
Carnegie Mellon University
USA
tahak@cmu.edu

8th Muhammad Ali Gulzar
Virginia Tech
USA
gulzar@cs.vt.edu

***Abstract*—Generative Large Language Models (LLMs) are increasingly used in non-generative software maintenance tasks, such as fault localization (FL). Success in FL tasks depends on a model's ability to reason about program semantics that are beyond surface-level syntactic and lexical features. However, widely used LLM benchmarks primarily evaluate code generation, which differs fundamentally from program semantic reasoning. Meanwhile, traditional fault localization benchmarks like Defect4J and BugsInPy are either not scalable or obsolete because their datasets have become part of LLM training data, leading to biased results. This paper presents the first *large-scale* empirical investigation into the robustness of LLMs' fault localizability. Inspired by mutation testing, we develop an end-to-end evaluation framework that addresses several limitations in current LLM evaluation, e.g., data contamination, scalability, automation, and extensibility.**

Given real-world seed programs with specifications, we inject unseen faults and ask LLMs to localize them. We filter out underspecified programs, where correct fault localization is inherently ambiguous. For each program an LLM localizes successfully, we apply semantic-preserving mutations (SPMs) and rerun localization to assess the LLM's robustness and whether the LLM's reasoning relies on syntactic cues rather than semantics. We evaluate 10 state-of-the-art LLMs on $750{,}013$ fault-localization tasks sourced from over 1300 Java and Python programs. We observe that SPMs cause an LLM to fail to localize the same fault it correctly localized earlier in 78% of cases, and that LLMs' reasoning on the code found earlier in the context is noticeably better. These results suggest that LLMs' code-reasoning is tied to code features irrelevant to semantics. We also identify code patterns that are challenging for LLMs to reason about. To the best of our knowledge, no prior work has evaluated the robustness of LLMs' code reasoning in fault localization at this scale. Overall, our findings motivate fundamental advances in how LLMs *represent*, *interpret*, and *prioritize* code semantics to reason more deeply about program logic.

***Index Terms*—Large Language Models, Fault Localization, Code Debugging**

I. Introduction

Large Language Models' (LLMs) growing adoption in software maintenance [1]–[3] requires code *reasoning* capabilities that differ fundamentally from code generation. In generation, models translate natural language descriptions into code, whereas software maintenance tasks demand understanding and operating on *existing* code, often to answer questions such as fault localization or bug diagnosis. Consequently, while code generation is widely benchmarked [1], [4], [5], limited benchmarking mechanisms exist for code reasoning tasks, and these studies typically suffer from data contamination, limited scalability, and insufficient rigor [6]–[8]. As a result, reported performance can be overly optimistic, and models may be overfitted to these benchmarks.

LLMs are increasingly used for fault localization (FL) [9]–[12]. Effective FL requires models to go beyond lexical or structural cues and reason about program *semantics*, e.g., how state flows, how components interact, and how logic gives rise to observed failures. There is a critical need for systematic assessment of LLMs' fault-localization capabilities for the future development of reliable coding models and autonomous agents. There are three key challenges in assessing LLMs' code reasoning for fault localization.

First, unlike traditional FL techniques that rely primarily on test suites, LLMs often require explicit specifications in addition to code to perform fault localization effectively. The absence of such specifications in many existing datasets, therefore, imposes additional constraints on evaluating LLMs. Second, prior work [9], [10], [13]–[15] frequently evaluate LLM-based fault localization on public benchmarks such as Defects4J [16] and BugsInPy [17]. Because these datasets are included in LLM pre-training corpora, the data is considered contaminated, leading to overly optimistic performance. Even with a newly constructed benchmark, LLMs undergo continuous training and are eventually exposed to public datasets [6]. Third, evaluating LLMs' code reasoning capabilities itself remains challenging due to the lack of standardized, scalable evaluation methodologies. Prior research has largely focused on human developers and relies on qualitative user studies, which do not readily translate to automated, large-scale evaluation of LLMs.

In this work, we conduct the first large-scale empirical investigation of LLMs' ability to reason about code for fault

localization. To do so, we design an automated evaluation framework that dynamically generates controlled, previously unseen fault localization tasks to evaluate both accuracy and the robustness of LLMs' code reasoning for fault localization. Our insight is that if an LLM can correctly reason about a program's semantics with respect to a specification, it should (a) identify deviations that change the intended behavior (i.e., faults) and (b) remain invariant to deviations that do not (semantic-preserving mutations). We address dataset contamination by dynamically injecting faults into real-world seed programs to create unseen fault localization tasks using classical mutation testing. For additional FL tasks, we apply semantic-preserving mutations (e.g., changes to comments and variable names, and the insertion of dead code) to faulty programs. These transformations preserve the original specification while generating additional FL tasks, thereby reusing it and avoiding the need to write it manually. This approach is analogous to fuzzing, where diverse variants are generated through controlled transformations.

Given a set of source programs and their specifications, we first dynamically inject simple fault widely used in prior mutation testing research [18], including operator mutations (e.g., ==, != , <), conditional logic changes (e.g., branch inversion), constant and boundary modifications (e.g., off-by-one errors), and incorrect variable or return-value substitutions [19], [20]. We prompt each LLM with the faulty program and its specification and ask it to identify the injected fault's line number. To avoid ambiguous FL tasks due to underspecified programs, we apply a counterexample-driven existential filter: we retain a (program, spec) instance if one or more LLM can localize the injected fault under that specification.

Since LLMs rely on attention mechanisms, they may overemphasize non-functional elements, such as comments or dead code, particularly when such patterns are frequent in the training data. We select only faulty programs correctly localized by the LLM and generate multiple semantic-preserving mutations (SPMs) that preserve program behavior and the injected fault to test the model's fault localization robustness. SPMs include identifier renaming, comment insertion or modification, formatting changes, and dead-code insertion, all of which are widely used in prior work [21], [22]. Each mutated program is paired with the exact same specification and provided to the LLM to re-identify the faulty line. By comparing fault localization responses across original, faulty, and mutated variants, we assess whether LLMs rely on true semantic cues or superficial code features.

We conduct our empirical investigation on state-of-the-art code datasets [23], [24], in which each program is accompanied by a specification describing its intended behavior. We start with 1,307 seed programs, comprising 637 Python and 670 Java programs. From these seeds, we inject 4 fault types and apply 6 semantic-preserving mutations, generating 750,013 unique faulty programs. The resulting dataset spans 245 million lines of code (LOC) and approximately 3.8 billion tokens. We evaluate 10 state-of-the-art LLMs, spanning both closed-source commercial models (e.g., Open AI [25], Claude [26]) and open-source models (e.g., qwen [27], phi4 [28]).

```
1  def solveNQueens(n):
2      def is_safe(board, row, col):
3          for i in range(col):
4              if board[row][i] == 1:
5                  return False
6
7          for i, j in zip(range(row, -1, -1),
8                          range(col, -1, -1)):
9              if board[i][j] == 1:
10                 return False
11
12         # Off-by-one fault: the loop stops one row too
    early
13         for i, j in zip(range(row, n-1, 1),
14                         range(col, -1, -1)):
15             if board[i][j] == 1:
16                 return False
17         return True
18     <CODE REMOVED FOR BREVITY>
19     def solveQueen(board, col, result):
20     <CODE REMOVED FOR BREVITY>
21 n = 4
22 solveNQueens(n)
```

Fig. 1: N-Queen program, P_F , with an injected fault.

Our investigation shows that even simple semantic-preserving mutations are sufficient to throw off LLMs' reasoning capabilities. Across faulty programs that an LLM initially localizes correctly, applying SPMs causes LLMs to fail to localize the same fault in 78% of cases. Further analysis reveals that misleading comments and dead code, both common in real-world programs, account for the majority of this robustness loss, with dead code alone reducing average accuracy to 20.38%. As expected, increasing the strength of SPMs produces a near-linear degradation trend: across all models, fault localization accuracy drops by 1.04% per mutation step in Java and 1.93% per step in Python, indicating compounding semantic interference. Our evaluation framework is extensible and highly configurable. By controlling fault location, we find that 56% of correctly localized faults appear within the first 25% of program lines, compared to only 6% in the final 25%, highlighting strong positional bias. We also track LLMs' evolution in code reasoning for fault localization. Newer Claude and Gemini variants show very modest fault localizability gains (1–2%), suggesting that recent model scaling and retraining have not substantially improved code reasoning required for FL.

To the best of our knowledge, this is the first empirical investigation of LLMs' code reasoning in fault localization tasks at this scale, supported by a fully automated, extensible evaluation framework. Our findings indicate that LLMs' code-reasoning in FL is highly nonlinear across the code surface and remains sensitive to code artifacts that are completely irrelevant to code semantics, thus motivating more fundamental advances in how LLMs *represent*, *interpret*, and *prioritize* code to reason more deeply about code semantics.

Data Availability. The evaluation framework, datasets, and code generated and analyzed during the study are publicly available on Zenodo: https://doi.org/10.5281/zenodo.18796544

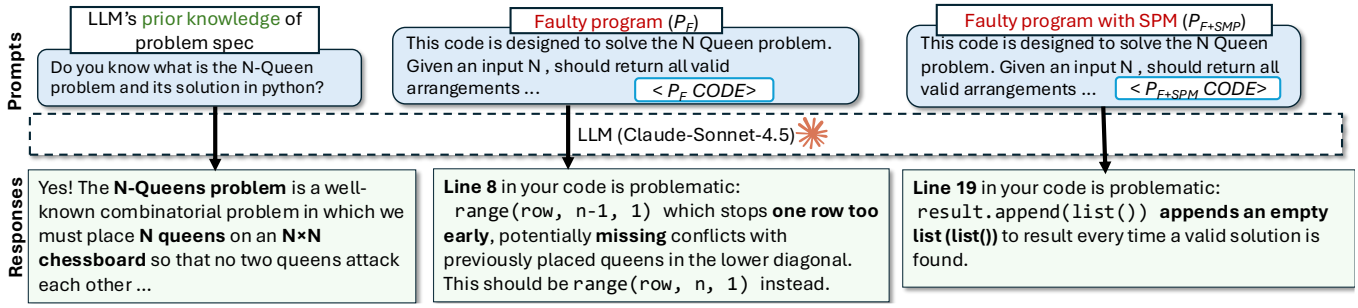


Fig. 2: Output of Claude's Sonnet 4.5 when asked about the N-Queen problem spec (first), asked for faulty line in the faulty code *(P_F)* (second), and asked for faulty line in the faulty code ***with*** semantic-preserving mutations *(P_{F+SPM})* (third) .

II. Motivating Example

We present a running example based on N-Queen problem taken from the Python dataset [23]. To illustrate the limitations of LLMs' code reasoning for fault localization, we inject a fault in this program and ask LLMs to identify the faulty line with and without semantic-preserving mutations. We use Claude Sonnet 4.5 in this example. The N-Queen problem involves placing N queens on an $N \times N$ chessboard so that no two queens threaten each other; that is, no two queens share the same row, column, or diagonal. This is a combinatorial optimization problem commonly solved using backtracking and constraint programming. We select the N-Queen problem because its algorithm is well-known, and its specification is unambiguous. We verify this by querying LLM about its specifications, which it accurately lists (Figure 2-Left).

For fault injection, we introduce an off-by-one fault in `is_safe` function in line 13 as shown in red text in Figure 1. The fault replaces `n` with `n-1`, which stops the exploration of the *lower diagonal* of the board. The lower diagonal refers to the set of cells that lie diagonally downward (i.e., increasing row indices) and to the left (i.e., decreasing column indices) from a given position. Thus, a queen positioned along this diagonal can attack another queen located at the current position. We call this program P_F. We create another version of this faulty program by injecting three semantic-preserving mutations: misleading function name, variable name, and comments, shown in Figure 3. Specifically, we rename the function name `solveNQueens(n)` to `howManyQueens(n)` and add a misleading comment (*"This function checks how many queens are on the board."*) and change the variable name `board` to `final_result`. We call this program P_{F+SPM}. Next, we prepare two FL tasks for LLM, by first providing P_F and then P_{F+SPM}, the original N-queen specification, and ask the LLM to identify the faulty line of code in both. The prompt given to the LLM for both versions is:

"This code is designed to solve the N-Queen problem. Given an input N, it should return all valid arrangements of N queens on an $N \times N$ board such that no two queens attack each other. However, the code produces incorrect output. Can you identify the specific line of code responsible for the error? The program is attached below. `<CODE>`*"*

```
1  Misleading variable name.
2  def howManyQueens(n):
3      def is_safe(final_result, row, col):
4          Misleading comment.
5          # This function checks how many queens
6          are on the board.
7          for i in range(col):
8              if final_result[row][i] == 1:
9                  return False
10
11         for i, j in zip(range(row, n-1, 1),
12                         range(col, -1, -1)):
13             if final_result[i][j] == 1:
14                 return False
15         return True
16     <CODE REMOVED FOR BREVITY>
17     def solveQueen(board, col, result):
18         if col == n:
19         result.append(list())
20     <CODE REMOVED FOR BREVITY>
21 Misleading function name.
22 howManyQueens(n)
```

Fig. 3: N-Queen program, P_{F+SPM}, with an injected bug in Line 11 and Semantic Preserving Mutations.

For the first fault localization task, we ask the LLM to localize the faulty line of code in the faulty program, P_F. It correctly identifies the faulty line of code, as shown in Figure2-Middle. To measure the robustness of LLM, we create the second fault localization task by asking LLMs to find the faulty line of code in the program, P_{F+SPM}. An in-depth, high-quality code reasoning would allow LLM to discard any changes that do not impact the code semantics while continuing to find the fault it identified earlier. However, as shown in Figure 2, LLM does not identify the faulty line of code in the presence of these SPMs. Instead, it erroneously flags line 19, `result.append(list())`, as problematic, while the off-by-one error in `is_safe` function remains undetected. While Claude Sonnet 4.5 correctly detects the fault with respect to the program's specification, its lack of robustness causes its code reasoning to be easily influenced by semantically irrelevant changes.

III. Research Questions

To systematically evaluate the accuracy, robustness, and evolution of LLMs' fault localizability, we investigate the following research questions.

1) **RQ1: Robustness of LLMs to Semantic-Preserving Mutations**: How robust are LLMs' fault localization abilities when programs are subjected to SPMs?
2) **RQ2: Effect of SPM Types and Strengths on Fault Localization**: How do different types and strengths of SPMs affect LLMs' fault localization performance?
3) **RQ3: Effect of Fault Location on LLMs' Fault Localization Ability**: How does the location of a fault within a program influence LLMs' ability to localize faults?
4) **RQ4: Differences Across LLM Categories**: How do different categories of LLMs differ in fault localization performance under SPMs?
5) **RQ5: Longitudinal Trends in LLM Fault Localization**: Do LLMs exhibit measurable improvements in fault localization robustness over time as models evolve?

IV. Methodology

The goal of this work is to dynamically generate an arbitrarily large number of fault localization tasks, with labeled, correct responses expected from LLMs, to test their accuracy and robustness in code reasoning for fault localization. To that end, we design an automated end-to-end evaluation framework to measure an LLM's fault localization ability, as shown in Figure 4. In this section, we explain the design of this evaluation framework.

Given a set of seed programs ❶, we automatically inject faults into the programs ❷ and ask LLMs to localize the faults ❸. We then exclude fault-localization tasks with underspecified programs, since failure to localize a fault in such programs is due to insufficient specification. Next, we evaluate LLMs on the remaining fault-injected programs by recording their fault localization performance. We then take the programs that an LLM previously localized correctly and automatically inject semantic-preserving mutations ❹ with varying strengths, types, and locations, thereby creating a large number of unique fault localization programs for which ground truth is available. Finally, we send these mutated fault localization programs to LLMs and record their responses ❺, capturing fault localization accuracy and identifying LLM robustness in the presence of non-functional code changes.

A. Seed Programs Procurement

We focus this empirical study on the two dominant languages, Python and Java, used in code generation benchmarks [29]. Both languages are widely used in open-source projects [30], [31], consequently providing adequate training opportunities for LLMs. Evaluating LLMs on these languages provides the best opportunity to demonstrate their performance, compared to languages with limited training data.

Dataset criteria. We use the following criteria to identify the seed programs. First, the programs must be accompanied by natural language specifications. Without specifying the expected semantics of the code, the notion of correctness/incorrectness for fault localization will remain unclear. Second, the program must have at least 50 lines of code. LLM benchmarks containing small ($<$ 50 LOC) toy programs do not represent real-world programs [32]. Third, we avoid using already faulty programs or fault benchmarks (e.g., Defects4J [16] or BugsInPy [17]), as prior work [6]–[8] suggests that these datasets have been seen during LLM training, leading to data contamination. Lastly, the complete size of each program should not exceed the prompt size limits of the current LLMs APIs. Our focus is not to perform adversarial attacks on LLMs by creating challenging prompt scenarios. Instead, we aim to identify systematic patterns in code characteristics that reveal weaknesses in LLM fault localization ability.

Dataset Selection. We find two public benchmarks of Python and Java that satisfy the criteria above. We obtain Python programs from [23] and Java programs from CodeSearchNet [24]. The dataset consists of 18612 Python programs and 812 Java programs comparable to top LLM benchmarks [33], [34]. After applying the size and context limit filter, we get the final set of 637 Python and 670 Java programs. Figure 5 presents the line-of-code (LOC) distribution of these programs.

B. Fault Injection and Localizability

We adopt faults from standard mutation testing research [35]. We focus on (1) faults that are confined to a single line, (2) faults pertaining solely to the logical elements of the top-level code and cannot stem from dependencies, and (3) faults that should affect the program's logic rather than introduce syntax errors. Since LLMs today process code sequentially, token-by-token, *their understanding of code may vary across different program locations*. To discover further evidence, we randomly selected program locations from the 0-25%, 25%-50%, 50%-75%, and 75%-100% lines of code to inject fault. Table I presents the types of injected mutations listed under the `Type fault-inducing` mutations. For instance, incorrect boolean logic and arithmetic operators swap are LOR and AOR mutation operators from MAJOR [35], whereas premature return and off-by-one are from [36], [37]. Our rationale for using simple, standard mutations is to avoid challenging, rare cases that intentionally conceal faults and to provide a fair evaluation setting. If LLMs struggle to localize faults under these standard mutations, they are likely to struggle even more under more challenging faults.

Filtering Under-specified Fault Localization Tasks. Before assessing LLMs' faulty localizability, we must ensure that the programs are not under-specified, as correct fault localization for such programs is ambiguous. We treat specification inadequacy as a falsifiable hypothesis and apply a counterexample-based existential criterion: if one or more LLMs localize the injected fault successfully, we retain that FL task as solvable under the given specification. This is shown in ❸ in Figure 4.

Fault Localizability. After fault injection and excluding under-specified FL tasks, we provide a set of ten LLM models with a prompt that includes the program, its natural-language specification, and the task. The ten LLMs, listed in Table III, are selected based on their popularity and include widely used proprietary and open-source alternatives. Open-source models were executed on local server machines with NVIDIA L40S GPU. Meanwhile, closed-source models were accessed

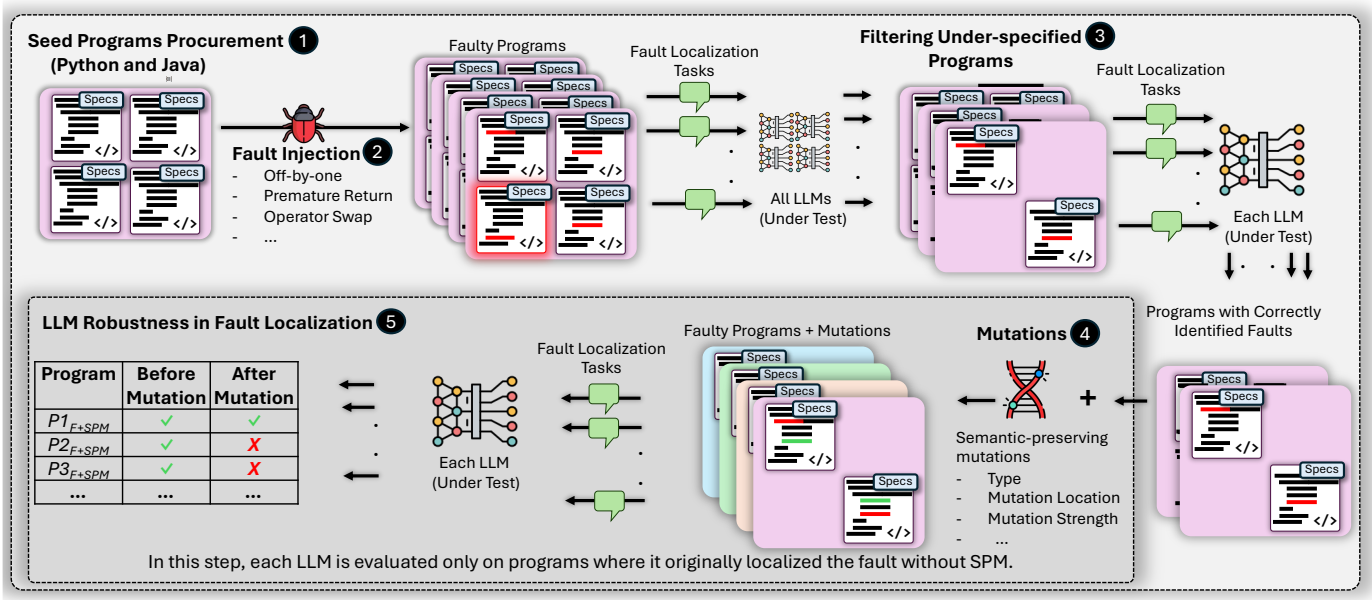


Fig. 4: Overview of our methodology, which consists of seed program procurement ❶, controlled fault injection ❷, filtering of under-specified programs via fault localization ❸, and robustness evaluation using semantic preserving mutations ❹ – ❺.

Modification	Type	Description	Example
Off-By-One	Fault-Inducing	Alters the loop range, causing an off-by-one error.	`for i in range(n)` → `for i in range(n+1)`
Misplaced Return	Fault-Inducing	Adds a return statement at an unintended location, leading to early termination.	`a=4; b=2+a;` → `a=4; return; b=2+a;`
Boolean Logic	Fault-Inducing	Switches boolean operators	`a && b` → `a || b`
Operator Swap	Fault-Inducing	Switches arithmetic operators	`a + b` → `a - b`
Dead Code Injection ($\mathcal{M}_d$)	Semantic-Preserving	Adds code that does not execute or is unused. Increases complexity without changing semantics.	`if(False): x = 5`
Misleading Comments ($\mathcal{M}_c$)	Semantic-Preserving	Replaces comments with misleading but coherent descriptions.	`/* Summon ancient dragons */`
Misleading Variable Names ($\mathcal{M}_v$)	Semantic-Preserving	Replaces variable names with ones that obscure their real function.	`count` → `index`
Function Shuffling ($\mathcal{M}_f$)	Semantic-Preserving	Shuffles the order of function definitions without breaking dependencies. Only on Java.	`void fA(){}; void fB(){}` → `void fB(){}; void fA(){}`

Table I: Types of Faults and Mutations Applied to Seed Programs

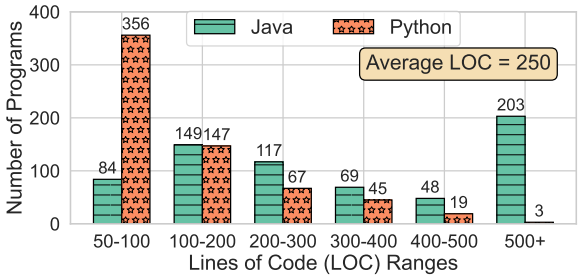


Fig. 5: LOC Distribution for the Final Set of Programs

via their respective APIs. Once the LLM identifies the line at fault, we automatically compare it with the recorded line number from the fault injection. We report and analyze the fault localization accuracy of these models on these FL tasks in Section V-A. If an LLM correctly identifies the fault, only then the faulty program is passed to the next step, where robustness is evaluated on the same LLM.

C. LLM's Robustness in Fault Localization

This step evaluates the robustness of the LLM's code reasoning by applying semantic-preserving mutations to faulty programs on which it previously localized faults correctly. Our insight is that if an LLM can adequately reason about code, it should be able to ignore semantically irrelevant changes that do not affect program functionality, thereby maintaining its fault-localization performance. The benefits of this process are threefold. First, these semantic-preserving mutations provide a stepwise dial to evaluate the limits of the LLM without manually writing new specifications, which can be infeasible at scale. Second, semantic-preserving mutations reaffirm that the LLM under test is not just syntactically comparing the program (i.e., a form of shallow reasoning) with a version it has seen in its training data. Lastly, evaluating robustness with the same LLM that solved the FL task confirms the specification is sufficient for the model to reason about the program and that the task is indeed solvable.

On programs that an LLM has successfully localized fault

Fault Type	Gemini 2.0 Flash	Gemini 1.5 Pro	GPT-4o	Llama 3.1	Phi-4	Qwen 2.5-coder	Qwen-QwQ	Claude-3.7-Sonnet	Gemini-2.5-Flash	Claude-4.5-Sonnet
Python Benchmark Programs										
IncorrectBooleanLogic	25.22	32.74	20.35	11.95	19.91	13.27	15.49	**36.28**	27.88	31.86
MisplacedReturn	40.08	29.60	23.25	9.31	23.96	11.14	14.29	44.76	35.40	**58.75**
OffByOne	30.13	31.45	32.65	16.33	19.45	17.53	16.69	33.49	30.61	**44.78**
OperatorSwap	29.07	**37.41**	28.12	15.81	22.25	12.41	14.02	36.55	35.13	32.86
Java Benchmark Programs										
IncorrectBooleanLogic	14.43	25.00	17.27	6.44	10.57	9.79	11.08	51.80	17.27	**71.13**
MisplacedReturn	36.37	39.63	23.34	6.87	15.77	8.50	15.77	11.16	31.20	**41.67**
OffByOne	37.50	33.33	16.67	8.33	8.33	4.17	4.17	70.83	16.67	**83.33**
OperatorSwap	14.50	28.73	27.91	10.81	13.27	4.24	19.43	**68.95**	20.79	31.74

Table II: Detection accuracy of different LLMs on various bug types in Python and Java (computed from valid programs).

Model	Size	Type
Qwen2.5-coder [38]	7 B	Open-source
Llama3.1 [39]	8 B	Open-source
Phi4 [40]	14 B	Open-source
Qwen-QWQ [41]	32 B	Open-source
GPT-4o [42]	Undisclosed	Closed-source
Claude 3.7 Sonnet [43]	Undisclosed	Closed-source
Gemini 2.0-Flash [44]	Undisclosed	Closed-source
Gemini 1.5-Pro [45]	Undisclosed	Closed-source
Gemini 2.5-Flash [46]	Undisclosed	Closed-source
Claude 4.5-Sonnet [47]	Undisclosed	Closed-source

Table III: LLMs Evaluated

before, we apply semantic preserving mutations of different characteristics (❹ in Figure 4), prepare a fault localization task, and ask the same LLM to localize the same fault again without any context of the previous fault localization task (❺ in Figure 4). We develop four categories of such mutations:

- **Annotative:** Changes to non-executing parts of the code, such as comments, annotations, or metadata. They help evaluate how much LLMs rely on annotations or documentation for reasoning.
- **Identifier:** Changes to the names of variables, functions, or other identifiers. This tests the resilience of LLMs and whether their reasoning is tied to concrete code structure rather than abstract.
- **Structural:** Changes to the program structure that do not modify functionality. For example, inserting unreachable statements. This helps assess if LLMs ignore semantically irrelevant code.
- **Non-additive:** Changes to the code order without introducing new content or changing semantics. Examples include changing the order of function declarations.

For these four categories, we develop one representative mutation for each, summarized in Table I under `Type` "Semantic-Preserving" category. For Dead Code Injection, Misleading Comments, and Misleading Variable Names, the core elements, e.g., the content of the comments, the names of the variables, and the snippets of dead code, are generated via the LLM under test. We insert these components via AST manipulation. During the insertions, we continuously track the movement of the original fault, allowing us to still reliably identify faulty lines, even if their position shifts due to formatting changes. Similar to fault injection, our rationale for using simple semantic-preserving mutations is to avoid challenging, rare cases that intentionally conceal faults or distort LLMs' reasoning under unreasonably high noise volume. If LLMs struggle to localize faults under simple SPMs, they will likely perform even worse under larger, more complex ones.

Mutation applications. We generate multiple program variants by applying mutations both individually and in combination. This means that given a program $\mathcal{P}$, we construct a total of six program mutants. Using the notation in Table I, four mutants are obtained by applying the individual mutations: $\mathcal{M}_c(\mathcal{P})$, $\mathcal{M}_v(\mathcal{P})$, $\mathcal{M}_v(\mathcal{P})$, $\mathcal{M}_f(\mathcal{P})$; and two more are obtained by composing: $\mathcal{M}_c(\mathcal{M}_v(\mathcal{P}))$ and $\mathcal{M}_c(\mathcal{M}_v(\mathcal{M}_d(\mathcal{P})))$. For each mutation application, we randomly select program location from 0-25%, 25%-50%, 50%-75%, and 75%-100% percentiles of the lines of code to apply the mutations. We also change the strength of the mutation, e.g., the number of lines of dead code. By layering mutations incrementally, we create a structured progression of fault localization difficulty.

LLMs robustness. The resulting fault localization tasks are sent to the LLM that correctly localized the fault in the previous step. Once a model returns the predicted line number, it is compared against the expected faulty line position. The robustness accuracies of the models are reported in Section V-A.

D. Evaluation Framework Implementation

We develop this evaluation framework in Python. Seed programs are provided as JSON files containing a natural-language specification and the corresponding source code. Both fault injection and semantic-preserving mutation can be provided as AST transformation scripts using existing templates.

Both proprietary LLMs' APIs and local models are supported. Local inference is enabled via the Ollama runtime, allowing models to be specified without code changes. The framework automatically manages prompt construction, structured response parsing, and result aggregation.

V. Results

We analyze the performance of the LLMs across various dimensions, including fault type, fault location, language, SPM type, SPM strength, SPM location, and LLM category. We also conduct a longitudinal study that tracks performance evolution across different versions of the same state-of-the-art models. We create a total of over 750K debugging tasks for these

LLMs, spanning 245 million lines of code, and 3.8 billion tokens. Table IV documents the experiment statistics.

Metric	Value
Total Prompts Generated	1,163,686
Total FL Tasks	750,013
Average LOC Tested	250 LOC
Total LOC Tested	245,856,641 LOC
Total Token Analyzed	3,881,577,500
System Specification	48 GB RAM, 48 cores with NVIDIA L40S GPU
APIs	Gemini, Anthropic, OpenAI

Table IV: Experimental Statistics

A. RQ1: Robustness of LLMs' Fault Localization

First, we analyze the fault localization accuracy of individual LLMs. This is to establish a baseline for observing how performance degrades as we test the robustness of LLMs via SPMs. We exclude under-specified FL tasks. Table II presents these results. Accuracy is the ratio of faulty programs for which the LLM successfully localizes faults to the total number of faulty programs that are not underspecified.

The Claude family of models achieves the best overall performance in fault localization across both languages. Except Claude, almost all models show better performance in Python than in Java. Certain faults are better detected by LLMs than others. For example, 7 out of 10 models show the best performance on *MisplacedReturn*. This is consistent with token-sequential processing by LLMs. A return statement changes which later tokens are interpreted as reachable, shifting attention and, in turn, the model's fault localization reasoning.

Open-Source vs. Closed-Source LLMs. Notably, Claude 4.5 Sonnet, Claude 3.7 Sonnet, and Gemini 1.5 Pro lead in fault detection performance, demonstrating higher accuracy of fault localization, followed closely by OpenAI's GPT-4o. The highest fault detection accuracies come from closed-source models, suggesting that proprietary LLMs benefit from extensive training data, deployments on hardware needed to host the full-scale models, and optimizations. Among open-source LLMs, Phi-4:14B outperforms other open-source models.

1) Fault Localization After Applying SPMs: This analysis measures the impact of semantic-preserving mutations (SPMs) on an LLM's fault localization by asking the LLM to localize the same fault in a mutated faulty program (P_{F+SPM}) that it has successfully localized in a faulty program (P_F).

Figure 7 summarizes the results averaged across all fault types and semantics-preserving mutations (SPMs). Accuracy is measured as the ratio of successful fault localization after adding SPMs to the faulty programs, step ❺ of the process. Almost all models exhibit a noticeable degradation in fault localization accuracy when SPMs are introduced. Open-source models, including Llama3.1:8B and Qwen variants, exhibit the most significant performance declines, whereas closed-source models such as Claude, GPT-4o, and Gemini demonstrate greater robustness.

> **Takeaway:** LLMs exhibit significant drops in fault localization accuracy under semantic-preserving mutations, revealing difficulty in distinguishing semantically irrelevant code features from semantically relevant ones.

2) Language-specific Performance under SPMs: Figure 7 shows the performance breakdown by language. The results show that LLMs struggle more to detect faults in Java than in Python across all models; with a wider gap for the Claude, GPT-4o, and Gemini models. This behavior can be attributed to the size and diversity of the training data available for these models, as well as the intrinsic structural differences between Java and Python [48]. Python's concise syntax and its widespread use in scripting and data science lead to more comprehensive coverage training [49]. Conversely, Java's verbosity and strict object-oriented nature can pose additional challenges, such as maintaining a large context for fault detection. Most models exhibit reduced performance on Java compared with Python.

> **Takeaway:** While LLMs initially achieve higher fault-localization accuracy on Java (without SPMs; Table II), applying SPMs makes it substantially harder for them to localize the same fault. As a result, the degradation on Java is markedly larger than on Python (Figure 7).

B. RQ2: Effect of Mutation Characteristics

We further decompose results by mutation type and strength.

1) Effect of SPM Type on LLM's Fault Localizability: Figure 6 presents the average fault localization accuracy across different LLMs for different types of SPMs. *Misleading Variable Names* causes the least impact (29.02% accuracy) on fault localization accuracy. One reason is that variable renaming preserves the structural and lexical integrity of code. Because LLMs learn statistical patterns from training data that contain variable names of all types, there is no single variable-naming convention; therefore, LLMs are relatively robust to renaming.

Misleading Comments result in lower average accuracy (25.63%) compared to *Misleading Variable Names* (29.02%), indicating that LLMs may rely on comments for code reasoning. *Dead Code* has a more substantial impact on accuracy, dropping LLMs' fault localization accuracy drops to 20.38%. For example, one of the programs simulates an autonomous car; the update method contains the injected fault where the car's vertical movement is adjusted with an incorrect offset (using `self.rect.y + = self.change_y − 1`). The program contains an unreachable dead code block that defines a function for debugging sensor values. This dead code changes the syntactic structure without affecting functionality, distracting the model and leading it to erroneously flag the sensor log call within the dead code block as the fault, rather than the movement logic.

2) Effect of Function Reordering on Bug Detection Accuracy: Function reordering mutations only apply to Java programs. We observe a significant accuracy drop of 83%, demonstrating that even without introducing any new code

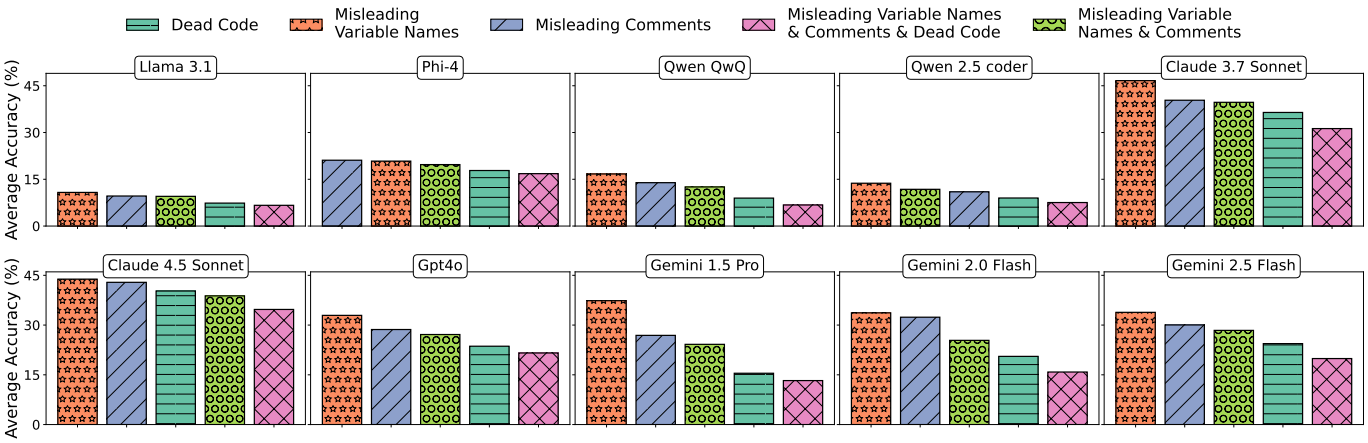


Fig. 6: Effect of mutation type on fault detection accuracy averaged for Python [23] and Java datasets [24].

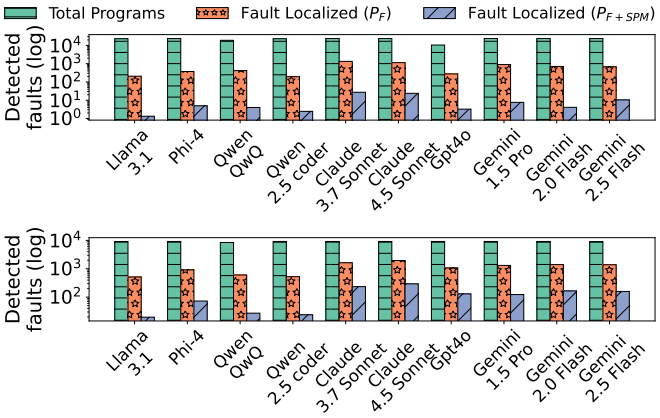


Fig. 7: Effect of mutations on fault detection accuracy across all fault types on each LLM with the Java dataset (top) and Python dataset (bottom).

or logic changes, the ability of LLMs to localize faults is compromised by structural shifts. This indicates that the physical location of code plays a prominent role in LLMs' code reasoning. Because Java is inherently more verbose than Python, these reordering mutations likely push critical logic into deeper parts of the context window where LLMs are known to exhibit reduced reasoning capacity [50], [51].

3) Effect of Mutation Strength: Figure 8 presents the results for each mutation type with varying strength levels (1 to 8), where strength represents the number of times an SPM was applied within a single program. For most LLMs, we observe a linear decline in average fault localization accuracy (indicated by the red line) as SPM strength increases. For instance, in Java, the accuracy drops on average from 15.82% at strength 1 to 8.57% at strength 8 across all evaluated models, corresponding to an overall decrease of 7.25% (an average decrease of 1.04% per mutation strength increase). Similarly, in Python, the accuracy falls, on average, from 42.88% at strength 1 to 29.34%, which represents an overall decline of 13.54% over 7 steps (an average decrease of 1.93% per mutation strength increase). In one seed Python program, the Blender API function `make_tile` is injected with an early `return None` fault. Gemini 1.5 Pro localizes the fault with low dead-code SPM strength, but fails when dead-code strength increases to 4 (multiple dead statements).

> **Takeaway:** Misleading comments and dead code cause the highest disruption in LLMs fault localizability, and even minor code reordering (e.g., function reordering) can reduce fault localization accuracy, highlighting LLMs' reliance on surface-level code cues.

C. RQ3: Effect of Fault Location in the Code

Figure 9 presents a heatmap summarizing fault localization accuracy across different fault types and their positions within the code. Results show that faults in the first quarter of the code ($0-25\%$) are detected with the highest accuracy, suggesting that LLMs may focus more on initial segments and retain clearer context at the beginning of a program. In particular, *OffByOne* faults are easily detected in this early section. However, detection accuracy declines as faults appear later in the code, with the $75-100\%$ range exhibiting the lowest success rates across all fault types.

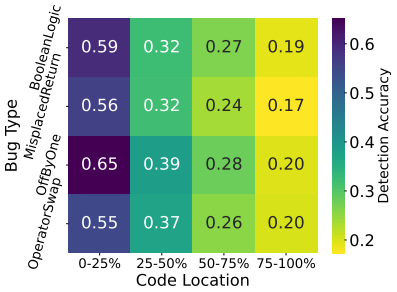
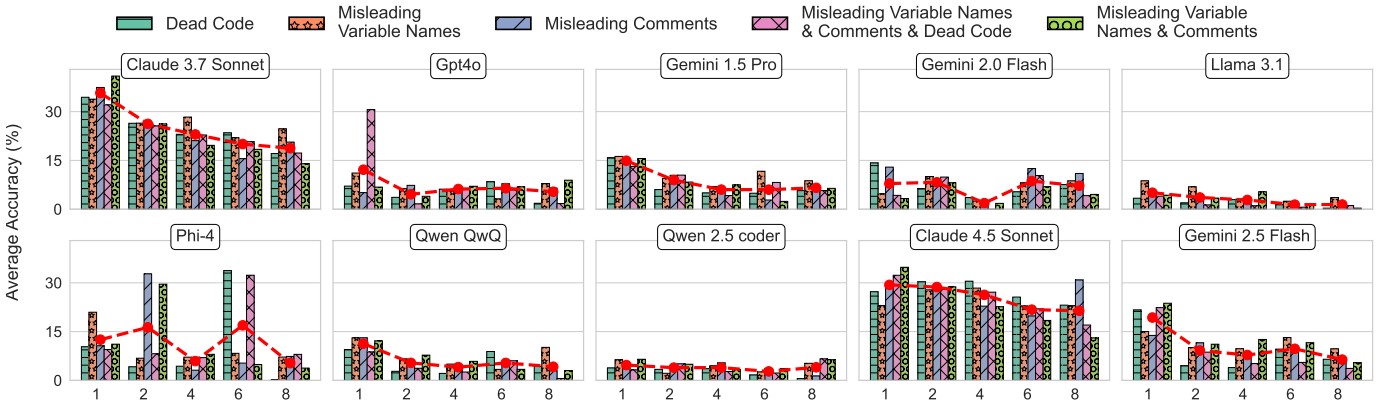
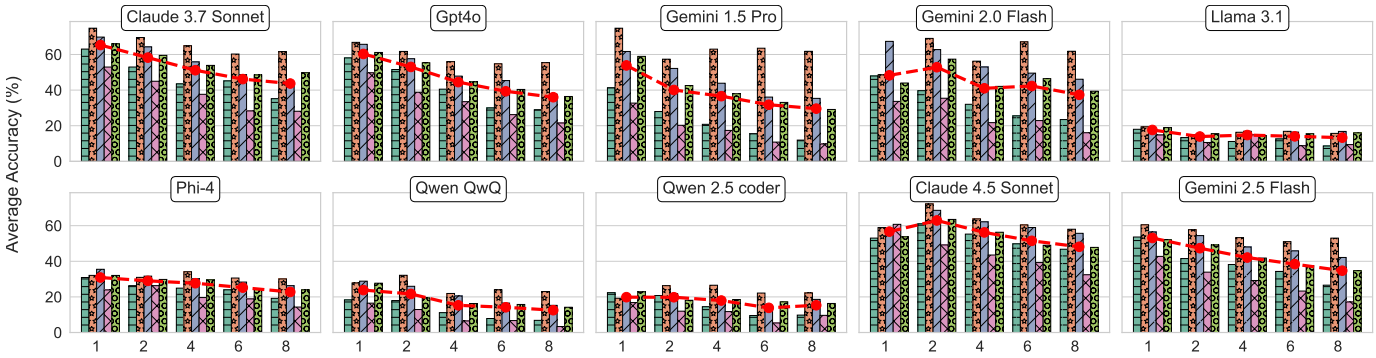


Fig. 9: Effect of fault location on fault detection accuracy.

These findings suggest that LLMs may lose context or allocate less attention to code segments appearing farther from the start due to the cumulative noise and attention decay inherent in long-sequence processing [52].



(a) Java programs



(b) Python programs

Fig. 8: Effect of mutation strength on fault detection accuracy by mutation type for different models. Red line shows the average accuracy across mutation strengths.

> **Takeaway:** LLMs are better at localizing faults in the early code regions, with accuracy declining for faults in later sections, indicating limited context retention and positional non-linear reasoning ability across code regions.

D. RQ4: Categories of LLMs

Figure 10 shows that reasoning and multimodal models like Claude and Gemini achieve the highest accuracy, while general-purpose models like Llama 3.1 and Phi-4, and coding-specialized models like Qwen2.5-coder:7B, yield the lowest performance. This discrepancy highlights a broader insight: LLMs optimized for reasoning may develop more robust internal representations of code semantics due to their exposure to diverse instruction-following and multi-step reasoning tasks, which enhances their ability to look past syntactic noise [53]–[55].

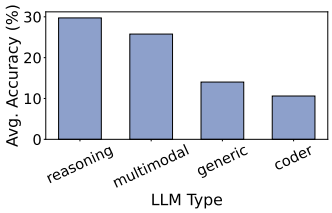


Fig. 10: Effect of LLM Type on fault detection accuracy.

E. RQ5: Longitudinal Study of LLMs

We also conduct a longitudinal analysis of the two best-performing model families, Gemini and Claude as shown in Figure 11. We observe a 1.8% improvement for Gemini and an average 1.0% improvement across Claude versions. These gains are consistent with expected model iteration effects like additional training data, improved training recipes for coding tasks, and overall capability improvements that translate to better code reasoning for fault localization. While directionally positive, these gains are marginal, underscoring the need for more fundamental advances in how models *represent*, *interpret*, and *prioritize* code semantics to reason more deeply about code logic.

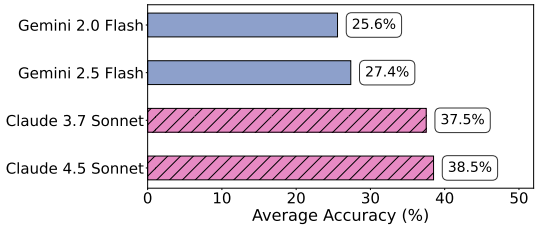


Fig. 11: Performance of the same models across versions.

VI. Discussions

In this section, we discuss key findings and evaluation design choices in an FAQ format.

What is the reason behind selecting single-file programs for such empirical investigation? We use simple, single-file programs with clear specifications to give LLMs the most favorable conditions. Even under those best-case settings, models fail to localize faults reliably. Theoretically, as program length grows, attention over many tokens leads to context

dilution and positional bias, making it harder to reason about code [56]. Thus, poor performance on simple programs provides a conservative lower bound.

What is the rationale for using semantic-preserving mutations? By design, classical fault-localization techniques (e.g., delta debugging [57] and mutation-based fault localization [58] and their extensions) rely on dynamic execution evidence such as tests and coverage, and are therefore robust to non-functional changes by SPMs. As LLMs are increasingly used as substitutes for these techniques in practice, developers expect comparable robustness. We thus argue that LLM-based fault localization should, at a minimum, meet the performance of these classical baselines. We employ simple, semantic-preserving mutations to incrementally increase the difficulty of fault localization tasks in a controlled way. This process is analogous to fuzzing, in which controlled transformations are systematically applied to the input to test the program.

Can prompt engineering improve LLMs fault localizability? We intentionally avoid extensive prompt engineering and use the *same* uniform baseline prompt illustrated in Figure 2. Although techniques such as interactive multi-shot prompting and iterative refinement might enhance performance, they require significant manual intervention, are unsuitable for large-scale evaluations, and overfit results to a single prompt.

What broader implications do our findings have on future directions in code representation? LLMs process code similarly to textual data, relying on generic tokenization methods that overlook code-specific features. We argue that converting code into intermediate, abstract, structured representations (like Control Flow Graphs (CFGs) and Code Property Graph (CPG) [59]) may enhance LLM reasoning capabilities. This approach can better capture syntactic and semantic nuances, improving robustness to non-functional code alterations.

VII. Threats to Validity.

Injected faults may not impact all possible execution paths or semantic behaviors. Future work could incorporate program analysis, such as program slicing or path-based fault injection, to improve execution path coverage. The evaluation includes ten LLMs. While this diversity mitigates model-specific bias, the results may not generalize to all existing or future LLMs. Additionally, our fault localization tasks are derived from Python and Java programs, and the findings may not extend to other programming languages. We select four injected fault types from prior mutation-testing literature. This set may not capture the full diversity of real bugs (e.g., multi-location bugs). To mitigate this, we select faults that are relatively easy to detect to provide a fair evaluation setting. Likewise, our selected SPMs do not cover all semantic-preserving transformations (e.g., refactorings, control-flow restructuring, API-equivalent rewrites). However, the simplicity of our SPMs provides a conservative lower bound on LLM robustness.

VIII. Related Work

Table V compares major benchmarks across five criteria.

Benchmark	GT	ML	CCC	Scal.	RDC
HumanEval+ [60]	✓	✗	✗	✗	✗
DebugBench [65]	✓	✓	✗	✗	✗
LiveCodeBench [63]	✓	✓	✗	✗	✓
SOAPFL [10]	✓	✓	✗	✗	✗
FlexFL [66]	✓	✓	✗	✓	✗
Ours	✓	✓	✓	✓	✓

Table V: Comparison of recent benchmarks under five criteria: **GT** (Ground Truth), **ML** (Multi-Language), **CCC** (Configurable Code Complexity), **Scal.** (Scalability), **RDC** (Robust to Data Contamination).

Code Generation and Benchmarking. LLMs for code are most commonly evaluated on *generation*-centric benchmarks such as HumanEval [1] and MBPP [4], along with follow-up suites that tighten functional evaluation e.g., HumanEval+ [60]. However, these benchmarks primarily measure whether a model can *produce* correct code, not whether it can reliably *operate on existing code* [61]. Recent work has also highlighted that public benchmarks can be contaminated by pretraining or post-release continual training [6]–[8], [62]. To mitigate this, "fresh" benchmark constructions such as LiveCodeBench [63] and LiveBench [64] sample tasks from post-cutoff time windows to reduce overlap with training data, but they still target generation performance rather than maintenance tasks like fault localization [63], [64].

Debugging and fault localization. Recent research [14], [67]–[69] utilizes and assesses the debugging capabilities of LLMs. They employ automated fault-localization strategies, including mutation testing and functional test cases, to determine whether an LLM can pinpoint errors; however, on standard fault benchmarks. Our work fills this gap by offering a mechanism to evaluate LLM-based FL methods on unseen FL tasks. [70] perform an evaluation of pre-trained models on code tasks using CodeXGLUE [5] and BigCloneBench [71] benchmarks, replicating results on tasks such as code search, generation, and defect detection, but their reliance on fixed datasets risks data contamination from pretraining corpora. Simiarly, [72] introduces ExeRScope, a toolkit for in-depth analysis of code execution reasoning frameworks (e.g., CodeMind [73], REval [74]) on benchmarks like CruxEval [33], also depend on static evaluation datasets that may overlap with LLM training data. [75] examine the internal validity of machine learning for vulnerability detection (ML4VD), which typically involves determining whether a given function is vulnerable without considering broader context. Their work primarily critiques the design of datasets and the framing of problems within ML4VD. In contrast, our work directly interrogates whether LLMs' *fault localization remains robust* under semantic-preserving code changes.

Program mutation and semantic analysis. Program mutation techniques have been used to stress-test model behavior under controlled transformations [76], [77]. We build on this idea but focus specifically on *fault-localization robustness* under semantic-preserving mutations. Similarly, other works [78]–

[81] have refined these techniques to model the effect of targeted semantic changes. Despite these advances, existing mutation frameworks typically emphasize semantic alterations that impact functionality while neglecting the subtleties of semantic-preserving changes.

IX. Conclusion

While LLMs are increasingly used in software development, their evaluation remains focused on code generation. This paper conducts a large-scale empirical study assessing LLMs' fault localization ability and their robustness under different scenarios. We automatically inject faults and semantic-preserving mutations in existing benchmarks to generate a large amount of unseen debugging tasks for LLMs. Experiments on 750K tasks reveal fundamental weaknesses in LLMs, with non-functional code changes reducing debugging accuracy by 78%, highlighting high sensitivity and low robustness under semantic-preserving mutations. We identify key code features that challenge LLMs and expose unique weaknesses, guiding research toward more effective LLM use for robust LLM-based debugging systems.

References

[1] M. Chen and et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] T. Li, Z. Wei, and M. Allamanis, "Can large language models reason about code?," *Empirical Software Engineering*, vol. 29, pp. 1234–1261, 2024.

[3] X. Wang, S. Fu, and N. A. Smith, "Large language models are human-level prompt engineers – but are they human-level program reasoners?," in *Proceedings of the 2025 IEEE/ACM International Conference on Software Engineering (ICSE '25)*, pp. 89–101, 2025.

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," 2021.

[5] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, L. Shou, L. Zhou, M. Tufano, M. Gong (YIMING), M. Zhou, N. Duan, N. Sundaresan, S. K. Deng, S. Fu, and S. Liu, "Codexglue: A machine learning benchmark dataset for code understanding and generation." arXiv, February 2021.

[6] C. Deng, Y. Zhao, X. Tang, M. Gerstein, and A. Cohan, "Investigating data contamination in modern benchmarks for large language models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (K. Duh, H. Gomez, and S. Bethard, eds.), (Mexico City, Mexico), pp. 8706–8719, Association for Computational Linguistics, June 2024.

[7] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. Le Goues, "Are large language models memorizing bug benchmarks?," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, pp. 1–8, 2025.

[8] Y. Dong, X. Jiang, H. Liu, Z. Jin, B. Gu, M. Yang, and G. Li, "Generalization or memorization: Data contamination and trustworthy evaluation for large language models," in *Findings of the Association for Computational Linguistics: ACL 2024* (L.-W. Ku, A. Martins, and V. Srikumar, eds.), (Bangkok, Thailand), pp. 12039–12050, Association for Computational Linguistics, Aug. 2024.

[9] A. Z. H. Yang, C. Le Goues, R. Martins, and V. Hellendoorn, "Large language models for test-free fault localization," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, (New York, NY, USA), Association for Computing Machinery, 2024.

[10] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao, "S¡sc¿oap¡/sc¿fl: A standard operating procedure for llm-based method-level fault localization," *IEEE Trans. Softw. Eng.*, vol. 51, p. 1173–1187, Feb. 2025.

[11] Cursor / Anysphere, Inc., "Cursor bugbot: Ai-powered code review and debugging tool." https://cursor.com/bugbot, 2025. AI-driven tool that automatically reviews pull requests and detects logic bugs, edge cases, and security issues in.

[12] N. Sekiyama, J. Zych, M. Saxena, R. A. Sharma, S. Mehta, S. Dsouza, V. Kajjam, W. Tang, and X. Yu, "Introducing generative AI troubleshooting for Apache Spark in AWS Glue (preview)." AWS Big Data Blog, Nov. 2024.

[13] S. Kang, G. An, and S. Yoo, "A quantitative and qualitative evaluation of llm-based explainable fault localization," *Proc. ACM Softw. Eng.*, vol. 1, July 2024.

[14] A. Nguyen and P. Brown, "Automated fault localization in llms challenges and techniques," in *Proceedings of the IEEE Conference on Software Engineering*, 2023.

[15] S. Ji, S. Lee, C. Lee, Y.-S. Han, and H. Im, "Impact of large language models of code on fault localization," in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pp. 302–313, 2025.

[16] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: a database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ISSTA 2014, (New York, NY, USA), p. 437–440, Association for Computing Machinery, 2014.

[17] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, "Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, (New York, NY, USA), p. 1556–1560, Association for Computing Machinery, 2020.

[18] D. Amalfitano, A. C. R. Paiva, A. Inquel, L. Pinto, A. R. Fasolino, and R. Just, "How do java mutation tools differ?," *Commun. ACM*, vol. 65, p. 74–89, Nov. 2022.

[19] V. Sobreira, T. Durieux, F. Madeiral, M. Monperrus, and M. Maia, "Dissection of a bug dataset: Anatomy of 395 patches from defects4j," 01 2018.

[20] A. V. Kamienski, L. Palechor, C.-P. Bezemer, and A. Hindle, "Pysstubs: Characterizing single-statement bugs in popular open-source python projects," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pp. 520–524, 2021.

[21] C. Na, Y. Choi, and J.-H. Lee, "DIP: Dead code insertion based black-box attack for programming language model," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (A. Rogers, J. Boyd-Graber, and N. Okazaki, eds.), (Toronto, Canada), pp. 7777–7791, Association for Computational Linguistics, July 2023.

[22] M. R. I. Rabin, N. D. Bui, K. Wang, Y. Yu, L. Jiang, and M. A. Alipour, "On the generalizability of neural program models with respect to semantic-preserving program transformations," *Information and Software Technology*, vol. 135, p. 106552, 2021.

[23] iamtarun, "Python code instructions 18k alpaca." https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca, 2023. Accessed: 2023-03-22.

[24] A. S. Soliman, "Codesearchnet dataset." https://huggingface.co/datasets/AhmedSSoliman/CodeSearchNet, 2023. Accessed: 2025-03-13.

[25] OpenAI, "Gpt-4 technical report," tech. rep., OpenAI, 2023.

[26] "Claude (language model)." Wikipedia, 2025. Claude is a family of large language models developed by Anthropic.

[27] "Qwen." Wikipedia, 2025. Qwen model family from Alibaba, open weights.

[28] M. Research, "Phi-4 technical report," 2024.

[29] H. Lee, S. Sharma, and B. Hu, "Bug in the code stack: Can llms find bugs in large python code stacks," 06 2024.

[30] D. Lu, J. Wu, Y. Sheng, P. Liu, and M. Yang, "Analysis of the popularity of programming languages in open source software communities," in *2020 International Conference on Big Data and Social Sciences (ICBDSS)*, pp. 111–114, 2020.

[31] L. Twist, J. M. Zhang, M. Harman, D. Syme, J. Noppen, H. Yannakoudakis, and D. Nauck, "A study of llms' preferences for libraries and programming languages," 2025.

[32] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Evaluating large language models in class-level code generation," in *Proceedings of the IEEE/ACM 46th International*

Conference on Software Engineering, ICSE '24, (New York, NY, USA), Association for Computing Machinery, 2024.

[33] A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang, "Cruxeval: a benchmark for code reasoning, understanding and execution," in *Proceedings of the 41st International Conference on Machine Learning*, ICML'24, JMLR.org, 2024.

[34] F. Tambon, A. Moradi-Dakhel, A. Nikanjam, F. Khomh, M. C. Desmarais, and G. Antoniol, "Bugs in large language models generated code: an empirical study," *Empirical Softw. Engg.*, vol. 30, Feb. 2025.

[35] R. Just, "The major mutation framework: efficient and scalable mutation analysis for java," in *International Symposium on Software Testing and Analysis*, 2014.

[36] H. Sellik, O. van Paridon, G. Gousios, and M. Aniche, "Learning off-by-one mistakes: An empirical study," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pp. 58–67, 2021.

[37] W. Shao, Y. Gao, F. Song, S. Chen, and L. Fan, "An empirical study of bugs in open-source federated learning framework," 08 2023.

[38] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024.

[39] A. Grattafiori and et al., "The llama 3 herd of models," 2024.

[40] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann, J. R. Lee, Y. T. Lee, Y. Li, W. Liu, C. C. T. Mendes, A. Nguyen, E. Price, G. de Rosa, O. Saarikivi, A. Salim, S. Shah, X. Wang, R. Ward, Y. Wu, D. Yu, C. Zhang, and Y. Zhang, "Phi-4 technical report," 2024.

[41] Q. Team, "Qwq-32b: Embracing the power of reinforcement learning," March 2025.

[42] A. Hurst and et al., "Gpt-4o system card," 2024.

[43] Anthropic, "Claude 3.7 sonnet and claude code," 2025.

[44] G. DeepMind, "Introducing gemini 2.0: Our new ai model for the agentic era," 2024. Accessed: March 14, 2025.

[45] G. DeepMind, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," 2024. Accessed: March 14, 2025.

[46] Google DeepMind, "Gemini 2.5 flash: Model overview." https://deepmind.google/technologies/gemini/, 2024. Accessed: 2025-01.

[47] Anthropic, "Claude 4.5 sonnet model card." https://www.anthropic.com/news/claude-4-5, 2024. Accessed: 2025-01.

[48] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, and A. Guha, "Multipl-e: A scalable and polyglot approach to benchmarking neural code generation," in *IEEE Transactions on Software Engineering*, IEEE, 2023.

[49] A. Smith, B. Johnson, and C. Lee, "Understanding the impact of training data diversity on code analysis," in *Proceedings of the IEEE/ACM International Conference on Software Engineering* (E. Gall and K. Schneider, eds.), vol. 1, pp. 150–160, IEEE, 2022.

[50] Y. Du, M. Tian, S. Ronanki, S. Rongali, S. Bodapati, A. Galstyan, A. Wells, R. Schwartz, E. A. Huerta, and H. Peng, "Context length alone hurts llm performance despite perfect retrieval," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 23281–23298, Association for Computational Linguistics, 2025.

[51] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[52] K. Hong, A. Troynikov, and J. Huber, "Context rot: How increasing input tokens impacts llm performance," tech. rep., Chroma, July 2025.

[53] D. Yang, T. Liu, D. Zhang, A. Simoulin, X. Liu, Y. Cao, Z. Teng, X. Qian, G. Yang, J. Luo, and J. McAuley, "Code to think, think to code: A survey on code-enhanced reasoning and reasoning-driven code intelligence in LLMs," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, eds.), (Suzhou, China), pp. 2586–2616, Association for Computational Linguistics, Nov. 2025.

[54] Y. Han, L. Xu, S. Chen, D. Zou, and C. Lu, "Beyond surface structure: A causal assessment of LLMs' comprehension ability," in *The Thirteenth International Conference on Learning Representations*, 2025. Accepted as a Poster at ICLR 2025.

[55] P. Shojaee*†, I. Mirzadeh*, K. Alizadeh, M. Horton, S. Bengio, and M. Farajtabar, "The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity," 2025.

[56] X. Wu, Y. Wang, S. Jegelka, and A. Jadbabaie, "On the emergence of position bias in transformers," 2025.

[57] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, p. 183–200, Feb. 2002.

[58] M. Papadakis, Y. L. Traon, F. Klein, and L. Seinturier, "Metallaxis-fl: Mutation-based fault localization," *IEEE Transactions on Software Engineering*, vol. 39, no. 10, pp. 1377–1396, 2013.

[59] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE Symposium on Security and Privacy*, pp. 590–604, 2014.

[60] J. Liu, C. S. Xia, Y. Wang, and L. ZHANG, "Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[61] D. Xie, M. Zheng, X. Liu, J. Wang, C. Wang, L. Tan, and X. Zhang, "Core: Benchmarking llms code reasoning capabilities through static analysis tasks," 2025.

[62] S. Chen, P. Pusarla, and B. Ray, "Dycodeeval: Dynamic benchmarking of reasoning capabilities in code large language models under data contamination," in *Forty-second International Conference on Machine Learning*, 2025.

[63] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "Livecodebench: Holistic and contamination free evaluation of large language models for code," in *The Thirteenth International Conference on Learning Representations*, 2025.

[64] C. White, S. Dooley, M. Roberts, A. Pal, B. Feuer, S. Jain, R. Shwartz-Ziv, N. Jain, K. Saifullah, S. Dey, Shubh-Agrawal, S. S. Sandha, S. V. Naidu, C. Hegde, Y. LeCun, T. Goldstein, W. Neiswanger, and M. Goldblum, "Livebench: A challenging, contamination-limited LLM benchmark," in *The Thirteenth International Conference on Learning Representations*, 2025.

[65] R. Tian, Y. Ye, Y. Qin, X. Cong, Y. Lin, Y. Pan, Y. Wu, H. Hui, W. Liu, Z. Liu, and M. Sun, "Debugbench: Evaluating debugging capability of large language models," 2024.

[66] C. Xu, Z. Liu, X. Ren, and D. Lo, "Flexfl: Flexible and effective fault localization with open-source large language models," *IEEE Transactions on Software Engineering*, 2025. Published.

[67] A. Smith and B. Johnson, "Advanced debugging techniques in llm based code generation," in *Proceedings of the ACM International Conference on Software Engineering*, 2023.

[68] S. Lee and D. Kim, "Deep debugging enhancing llms for robust code analysis," in *Proceedings of the ACM Conference on Software Engineering*, 2022.

[69] M. Hort, L. Vidziunas, and L. Moonen, "Semantic-preserving transformations as mutation operators: A study on their effectiveness in defect detection," 2025.

[70] Z. Zeng, H. Tan, H. Zhang, J. Li, Y. Zhang, and L. Zhang, "An extensive study on pre-trained models for program understanding and generation," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2022, (New York, NY, USA), p. 39–51, Association for Computing Machinery, 2022.

[71] J. Svajlenko and C. K. Roy, *BigCloneBench*, pp. 93–105. Singapore: Springer Singapore, 2021.

[72] C. Liu and R. Jabbarvand, "A tool for in-depth analysis of code execution reasoning of large language models," in *Companion Proceedings of the 33rd ACM Symposium on the Foundations of Software Engineering*, FSE '25, (New York, NY, USA), pp. 1–5, ACM, 2025.

[73] C. Liu, Y. Chen, and R. Jabbarvand, "Codemind: Evaluating large language models for code reasoning," *arXiv preprint arXiv:2402.09664*, 2024.

[74] R. Gupta, C. Orasan, and J. van Genabith, "Reval: A simple and effective machine translation evaluation metric based on recurrent neural networks," in *Proceedings of the 2015 conference on empirical methods in natural language processing*, pp. 1066–1072, 2015.

[75] N. Risse, J. Liu, and M. Böhme, "Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection," *arXiv Preprint*, vol. arXiv:2408.12986v2, 2024.

[76] C. Garcia and M. Rodriguez, "Ast based mutation testing for evaluating code semantics," in *Proceedings of the IEEE Software Engineering Conference*, 2023.

[77] M. Johnson and L. Clark, "Ast transformations for robust code analysis," in *Proceedings of the ACM International Conference on Software Engineering*, 2023.

[78] R. Patel and L. Wang, "Refining semantic alterations in code via ast transformations." arXiv preprint arXiv:2308.03873, 2023.
[79] D. Wong and E. Chen, "Evaluating semantic preserving mutations in code." arXiv preprint arXiv:2402.14261, 2024.
[80] A. Johnson and B. Lee, "A structural approach to llm code understanding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[81] M. Garcia and J. Doe, "Enhancing llms for code debugging via advanced mutation techniques," in *Proceedings of the ACM International Conference on Software Engineering*, 2023.